

Import Required Libraries

Students should:

- Import libraries for:
 - o Data handling
 - o Model building
 - o Tokenization
 - o Training
 - o Evaluation
- Briefly explain the purpose of each imported library.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from transformers import Trainer, TrainingArguments

from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix
```

Load and Explore Dataset

Students must:

- Load the selected dataset.
- Display sample records.
- Identify:
 - o Input text column
 - o Label column
- Perform class distribution analysis.
- Comment on whether the dataset is balanced or imbalanced.

```
dataset = load_dataset("imdb")
train_dataset = dataset["train"].select(range(1000))
test_dataset = dataset["test"].select(range(500))
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as `HF_TOKEN` in your Colab secrets, and restart this notebook.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md:      7.81k/? [00:00<00:00, 707kB/s]

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
plain_text/train-00000-of-00001.parquet: 100%          21.0M/21.0M [00:01<00:00, 105MB/s]

plain_text/test-00000-of-00001.parquet: 100%          20.5M/20.5M [00:00<00:00, 102MB/s]

plain_text/unsupervised-00000-of-00001.p(...): 100%    42.0M/42.0M [00:01<00:00, 109MB/s]

Generating train split: 100%                          25000/25000 [00:00<00:00, 49083.33 examples/s]

Generating test split: 100%                           25000/25000 [00:00<00:00, 87715.75 examples/s]

Generating unsupervised split: 100%                   50000/50000 [00:00<00:00, 144884.12 examples/s]
```

Text Tokenization and Preprocessing

Students must:

- Load a pre-trained tokenizer (e.g., BERT tokenizer).
- Apply:
 - o Tokenization

- o Padding to fixed sequence length
- o Truncation of long sequences
- Convert dataset into model-compatible format.

```
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")

def tokenize_function(example):
    return tokenizer(example["text"], padding="max_length", truncation=True)

train_dataset = train_dataset.map(tokenize_function, batched=True)
test_dataset = test_dataset.map(tokenize_function, batched=True)
```

```
config.json: 100%                               483/483 [00:00<00:00, 43.0kB/s]
tokenizer_config.json: 100%                     48.0/48.0 [00:00<00:00, 4.53kB/s]
vocab.txt: 232k/? [00:00<00:00, 859kB/s]
tokenizer.json: 466k/? [00:00<00:00, 1.05MB/s]
Map: 100%                                       1000/1000 [00:01<00:00, 723.56 examples/s]
Map: 100%                                       500/500 [00:00<00:00, 616.81 examples/s]
```

Prepare Dataset for Training

Students must:

- Rename label column appropriately (if required).
- Convert dataset format to PyTorch tensors.
- Ensure dataset is ready for model training.

```
train_dataset = train_dataset.rename_column("label", "labels")
test_dataset = test_dataset.rename_column("label", "labels")

train_dataset.set_format("torch", columns=["input_ids", "attention_mask", "labels"])
test_dataset.set_format("torch", columns=["input_ids", "attention_mask", "labels"])
```

```
model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
    num_labels=2
)
```

```
model.safetensors: 100%                       268M/268M [00:02<00:00, 202MB/s]
Loading weights: 100%                         100/100 [00:00<00:00, 466.41it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight]
DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key | Status |
-----+-----+
vocab_projector.bias | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |
vocab_transform.weight | UNEXPECTED |
vocab_transform.bias | UNEXPECTED |
vocab_layer_norm.bias | UNEXPECTED |
pre_classifier.bias | MISSING |
classifier.bias | MISSING |
classifier.weight | MISSING |
pre_classifier.weight | MISSING |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task
```

Train-Test Split

Students must:

- Use existing train/test split (if available), OR
- Perform manual train-test split.
- Justify the chosen split ratio.
- Explain the importance of separating training and testing data.

```
training_args = TrainingArguments(
    output_dir="./results",
```

```
eval_strategy="epoch",
learning_rate=2e-5,
per_device_train_batch_size=8,
num_train_epochs=1
)
```

Load Pre-trained Transformer Model

Students must:

- Load a pre-trained model (e.g., BERT).
- Specify number of output labels

Define Training Configuration

Students must:

- Define training arguments including:

- o Learning rate
- o Batch size
- o Number of epochs
- o Evaluation strategy

- Implement evaluation metrics:

- o Accuracy
- o Precision
- o Recall
- o F1-score

```
def compute_metrics(pred):
    logits, labels = pred
    predictions = np.argmax(logits, axis=1)

    precision, recall, f1, _ = precision_recall_fscore_support(labels, predictions, average='binary')
    acc = accuracy_score(labels, predictions)

    return {"accuracy": acc, "f1": f1, "precision": precision, "recall": recall}
```

Train the Model

Students must:

- Initialize the Trainer (or custom training loop).
- Train the model.
- Record:
 - o Training loss
 - o Evaluation metrics per epoch

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)

trainer.train()
```

[125/125 00:56, Epoch 1/1]

Epoch	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall
1	No log	0.002681	1.000000	0.000000	0.000000	0.000000

Writing model shards: 100%

1/1 [00:00<00:00, 1.70it/s]

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defir
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: F-score is ill-defi
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
TrainOutput(global_step=125, training_loss=0.04544932556152344, metrics={'train_runtime': 57.8206,
'train_samples_per_second': 17.295, 'train_steps_per_second': 2.162, 'total_flos': 132467398656000.0, 'train_loss':
0.04544932556152344, 'epoch': 1.0})
```

Model Evaluation

Students must evaluate the trained model using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Students must:

- Plot confusion matrix.
- Interpret the results.
- Comment on model performance.

```
predictions = trainer.predict(test_dataset)
y_pred = np.argmax(predictions.predictions, axis=1)
y_true = predictions.label_ids
cm = confusion_matrix(y_true, y_pred)

sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defir
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: F-score is ill-defi
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:407: UserWarning: A single label was found in 'y_
warnings.warn(
```



